

WHAT DP IS

// Break a problem into overlapping subproblems, solve each ONCE, store the
// result (memoize), reuse it instead of recomputing. Two forms: top-down
// (recursion + memo) and bottom-up (iterative table fill).

1D DP – TOP-DOWN (memoization)

Use when: recursive structure is natural but overlapping calls repeat work.

```
vector<long long> memo(n+1, -1);
long long solve(int i) {
    if (i <= 0) return baseCase;
    if (memo[i] != -1) return memo[i]; // already computed
    return memo[i] = combine(solve(i-1), solve(i-2) /* etc */);
}
```

1D DP – BOTTOM-UP (tabulation)

Use when: order of subproblems is clear – usually faster, avoids recursion overhead.

```
vector<long long> dp(n+1);
dp[0] = base0; dp[1] = base1;
for (int i = 2; i <= n; i++) dp[i] = combine(dp[i-1], dp[i-2]);
```

DEMO 1: FIBONACCI

```
vector<long long> dp(n+1); dp[0]=0; dp[1]=1;
for (int i=2;i<=n;i++) dp[i] = dp[i-1] + dp[i-2];
```

DEMO 2: CLIMBING STAIRS (1 or 2 steps at a time)

```
vector<long long> dp(n+1); dp[0]=1; dp[1]=1; // dp[i] = ways to reach step i
for (int i=2;i<=n;i++) dp[i] = dp[i-1] + dp[i-2];
```

DP ON ARRAYS

Use when: dp[i] depends on a[i] plus one or more previous dp states – 'take or skip' shape.

```
vector<long long> dp(n);
dp[0] = a[0];
for (int i=1;i<n;i++) dp[i] = max(dp[i-1], dp[i-1] + a[i]); // generic take/extend pattern
```

DEMO 3: MAXIMUM SUM / HOUSE ROBBER (no two adjacent chosen)

```
vector<long long> dp(n);
dp[0] = a[0]; dp[1] = max(a[0], a[1]);
for (int i=2;i<n;i++) dp[i] = max(dp[i-1], dp[i-2] + a[i]);
// answer = dp[n-1] // dp[i] = best using first i+1 houses
```

SPACE OPTIMIZATION NOTE

```
// if dp[i] only needs dp[i-1] and dp[i-2], drop the array – keep 2 variables
long long prev2=base0, prev1=base1, cur;
for (int i=2;i<=n;i++) { cur = prev1 + prev2; prev2 = prev1; prev1 = cur; }
```

// ===== HOW TO RECOGNIZE A DP PROBLEM =====

// - "count the number of ways to..." → likely DP

// - "find min/max cost/sum/length to..." → likely DP

// - Brute force is exponential AND subproblems repeat → DP candidate

// - Can you define dp[i] = "best answer using first i elements"? → good sign

// ===== DP CHECKLIST =====

// 1. Define the state: what does dp[i] represent?

// 2. Find the transition: how does dp[i] relate to smaller states?

// 3. Base case: what is dp[0] (or dp[1])?

// 4. Order: fill dp left-to-right, or need recursion + memo?

// 5. Answer: is it dp[n], max(dp), or something derived from the array?